

3D Object Recognition

1. Experiment Objective

Use MediaPipe Objectron to detect 3D objects (shoe, chair, cup, camera) in real time and draw 3D bounding boxes and coordinate axes.

2. Experiment Principle

1. Key Terms

Keyword	Description
MediaPipe Objectron	A 3D object detection model that outputs both the object's 3D bounding box and pose without requiring a depth camera
3D bounding box	The 3D bounding box of the object, composed of 9 vertices (8 corners + 1 center), projected onto the 2D image for drawing
Coordinate axes	RGB three-color axes (red=X, green=Y, blue=Z) indicating the object's spatial orientation
Four model types	Supports four pretrained models — shoe, chair, cup and camera — switchable at runtime

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start 3D object recognition
```

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒ Supported
No-PTZ version	☒ Supported
Required peripherals	ESP32 camera, and one of the four supported objects (shoe/chair/cup/camera)

2. Start the agent + camera + joint models (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```

yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully

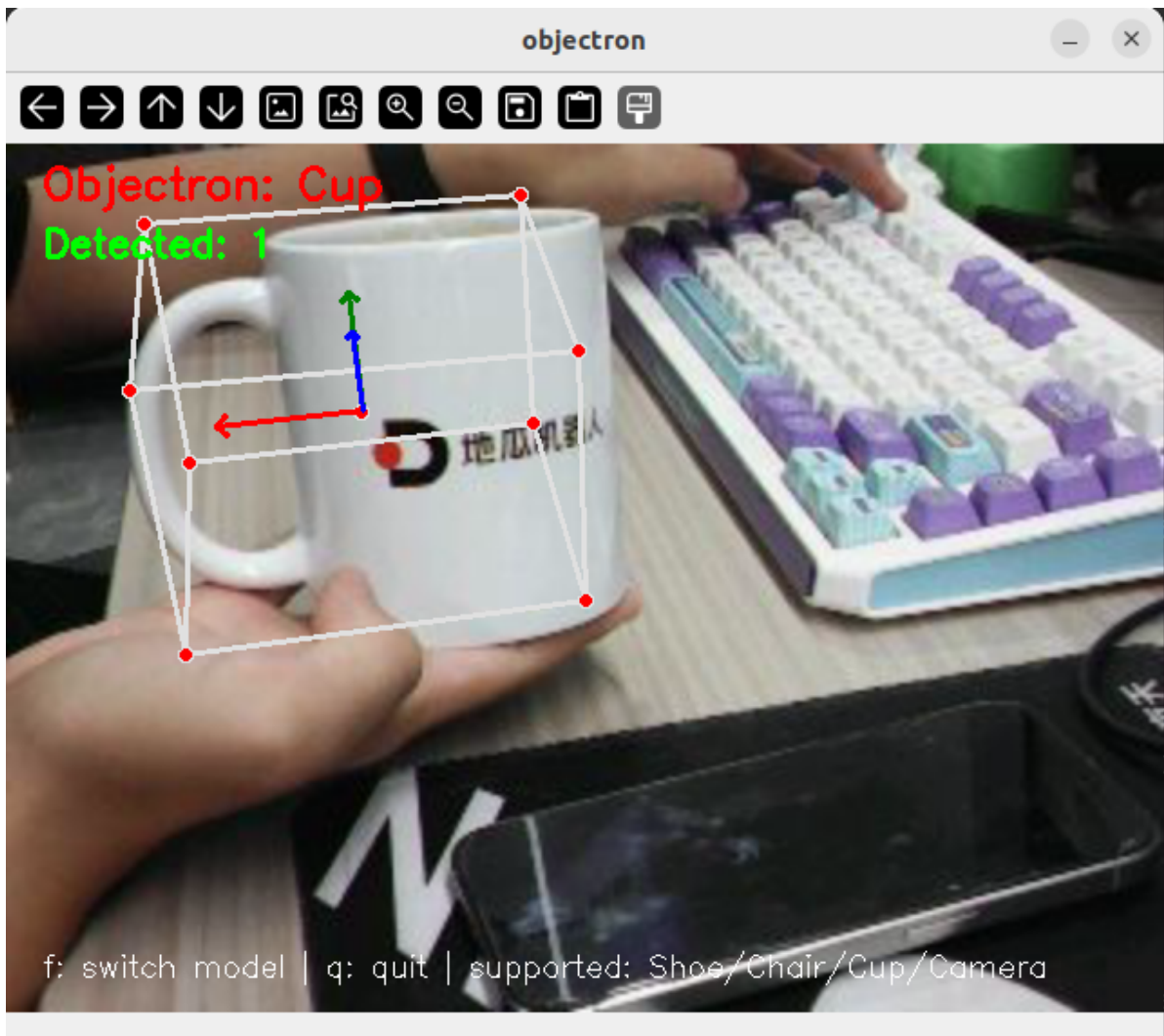
```

3. Start 3D object recognition (Terminal 2)

```
ros2 launch microros_v2_mediapipe_pkg objectron.launch.py
```

4. Observed Results

- Press **F** to switch among the four object types to detect
- Place one of the four objects in the camera's field of view and the object is enclosed by a 3D bounding box
- The RGB three-color axes indicate the object's spatial orientation (red=X, green=Y, blue=Z)



5. Code Walkthrough

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_mediapipe_pkg/launch/objectron.launch.py`
- Node:
`~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/Objectron.py`

1. Core Node Analysis

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/Objectron.py
class Objectron(Node):
    def __init__(self):
        super().__init__("objectron_node")
        self.declare_parameter("static_image_mode", False)
        self.declare_parameter("max_objects", 1)
        self.declare_parameter("detection_con", 0.3)
        self.declare_parameter("process_every_n", 3)
        self.declare_parameter("object_name", "Shoe")

        self.mp_objectron = mp.solutions.objectron
        object_model = getattr(self.mp_objectron.ObjectModel, object_name)
        self.objectron = self.mp_objectron.Objectron(
            static_image_mode=False, max_num_objects=1,
            min_detection_confidence=0.3, min_tracking_confidence=0.3,
            model_name=object_model)

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        results = self.objectron.process(cv.cvtColor(frame, cv.COLOR_BGR2RGB))
        if results.detected_objects:
            for obj in results.detected_objects:
                self.mp_draw.draw_landmarks(frame, obj.landmarks_2d,
                    self.mp_objectron.BOX_CONNECTIONS)
                self.mp_draw.draw_axis(frame, self.camera_matrix,
                    self.dist_coeffs, obj.rotation, obj.translation)
            cv.imshow(self.window_name, frame)
```

Key logic:

- `getattr(self.mp_objectron.ObjectModel, object_name)`: dynamically fetches the enum value from one of the four model names
- `process_every_n=3`: processes one frame out of every 3 to reduce computational load
- `min_detection_confidence=0.3`: the default threshold is lower than for Hands/Pose because 3D object detection is more difficult
- `draw_axis`: draws the RGB three-color axes; requires camera intrinsics (loaded from `camera.yaml`)
- Objectron tends to SIGSEGV under the Ubuntu VM's SVGA3D/EGL stack, so the startup script sets `LIBGL_ALWAYS_SOFTWARE=1` to force software rendering

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_mediapipe_pkg/launch/objectron.launch.py`
- Node:
`~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/Objectron.py`

Parameter	Type	Default	Description
<code>object_name</code>	string	<code>Shoe</code>	Detection model: <code>Shoe</code> / <code>Chair</code> / <code>Cup</code> / <code>Camera</code>
<code>max_objects</code>	int	1	Maximum number of objects to detect
<code>detection_con</code>	float	0.3	Detection confidence threshold
<code>process_every_n</code>	int	3	Process once every N frames (to reduce load)

3. Node Description

Node	Function
<code>objectron_node</code>	MediaPipe Objectron 3D detection + bounding box / coordinate axis visualization

6. Frequently Asked Questions

Issue	Cause	Solution
Object not detected	The object is not within the current model's supported scope	Switch <code>object_name</code> to the matching model
Display freezes in the VM	SVGA3D/EGL graphics stack compatibility issue	The script already sets the software rendering environment variable